AUDITORIA DE SEGURANÇA

Relatório de Auditoria de Segurança — Distributed Wagering Processor

Data: 01 de novembro de 2026

Situação após correções em 01 de novembro de 2026

Escopo

Auditoria da aplicação em `app/`, com dois auditores independentes (A e B) cujos achados foram consolidados neste relatório.

Auditor A	TypeScript estrito (Bun 1.4.4) · NestJS 11 (Express), nestjs-pino, @nestjs/swagger · MikroORM (PostgreSQL) usado só como executor; todo SQL é explícito em src/infrastructure/persistence/repositories com placeholders ? Autenticação: Nenhuma. AuthGuard é no-op (src/interfaces/http/auth.guard.ts:3-9) e NoopProviderIdentity não é chamado; decisão documentada em ARCHITECTURE §14 · Front-end: Sem SPA. Apenas apresentação estática (docs/apresentacao/index.html) e Swagger UI em /docs Deploy: docker-compose (app em 0.0.0.0, portas 3001-3003 no host), PostgreSQL 17 com papéis wagering_owner/wagering_app, MiniStack (SQS), Prometheus/Grafana/Jaeger (profile observability)
Auditor B	TypeScript estrito sobre Bun 1.4.4 · NestJS 11 (platform-express) · MikroORM 6.6.17 com SQL explícito, PostgreSQL 17.6 Autenticação: inexistente por decisão documentada (AuthGuard no-op, ARCHITECTURE §14) · Front-end: apresentação estática docs/apresentacao/index.html (JS puro), Swagger UI em /docs, dashboards Grafana provisionados Deploy: docker-compose.yml, docker-compose.test.yml, Dockerfile oven/bun:1.4.4-alpine, GitHub Actions ci.yml, perfil observability: Prometheus, Jaeger, Grafana

Nota metodológica

Categoria 1 — Banco sem tranca

Auditor A: Não existe usuário nem tenant. O único eixo de escopo é providerId, enviado pelo cliente no body (POST /wagering/transactions) ou no path (GET /providers/:providerId/...). Verifiquei se o mecanismo de isolamento existe: a tabela wallets não tem provider_id (Migration20261031000001_initial.ts:15-27), as queries de wallet/ledger filtram só por wallet_id, e providerId só entra nas chaves de idempotência e de id externo. Não há endpoint de listagem ou busca (nenhum GET devolve coleção além do ledger paginado de UMA wallet), então não há query de listagem sem filtro. O ledger filtra por wallet_id (ledger.repository.ts:34-37).

Auditor B: Fora do escopo do Auditor B (categoria do Auditor A).

Categoria 2 — Permissão definida no navegador

Auditor A: Não se aplica no sentido clássico: não há frontend com permissões, papéis ou flags no navegador. O Swagger UI só documenta e executa as mesmas chamadas HTTP; a apresentação é HTML estático sem lógica de autorização. Nenhuma decisão de acesso é tomada no cliente porque nenhuma é tomada em lugar algum (ver A-01).

Auditor B: Fora do escopo do Auditor B (categoria do Auditor A).

Categoria 3 — IDOR

Auditor A: Percorri os 8 handlers HTTP com ID no path/query/body (wagering.controller.ts: 3 rotas; wallets.controller.ts: 4 rotas) mais health e metrics. Nenhum tem checagem de posse, pois não há identidade a comparar (guard no-op). Os IDs são UUID validados por ParseUUIDPipe (wallets e transactionId) ou strings livres (providerId, externalTransactionId).

Auditor B: Fora do escopo do Auditor B (categoria do Auditor A).

Categoria 4 — Chaves expostas

Auditor A: Fora do meu escopo (Auditor B).

Auditor B: Busca por regex (postgres://, password, secret, token, api key, AKIA, PRIVATE KEY, admin, owner/app) em todo o repositório rastreado, exceto node_modules e bun.lock: src, scripts, test, infra, compose, Dockerfile, .env.example, .github, docs, README, ARCHITECTURE. Leitura integral de config.ts, mikro-orm.config.ts, compose, init-roles.sql, init-sqs.sh, Dockerfile, .dockerignore e .gitignore. Histórico: git log -p --all (89 commits) filtrado por AKIA, BEGIN PRIVATE KEY, password/secret/token/api_key, ghp_, xox, sk-; git log --name-only por arquivos .env/.pem/.key. Resultado: nenhum segredo real; só credenciais padrão de desenvolvimento. O único arquivo .env* no histórico é .env.example.

Categoria 5 — XSS

Auditor A: Fora do meu escopo (Auditor B).

Auditor B: Frontend: busca por innerHTML, outerHTML, insertAdjacentHTML, document.write, eval, new Function, setTimeout com string, srcdoc, javascript:, onclick=, href/src dinâmico em index.html. A tooltip do slide 12 usa textContent (linha 1131), logo não interpreta HTML. O único uso de location é location.hash casado por regex ^#slide-(\d+)$ (linha 999). Os 3 links são estáticos e relativos. Swagger: openapi.ts só define DTOs estáticos e o setup do SwaggerModule; a UI é o pacote do Nest, sem template próprio. Dashboards Grafana: wagering.json só tem expr PromQL e legendFormat fixos, sem links externos. Backend: nenhum motor de template ou HTML na resposta (todas as respostas são JSON ou text/plain); mensagens de erro do zod ecoam nomes de chaves (ver B-10). Não existe lib de sanitização (helmet, DOMPurify, xss, escape) em package.json nem em src. Como a API só emite JSON e não há HTML dinâmico, não há XSS refletido ou armazenado aplicável hoje.

Categoria extra — Testes extras

Auditor A: SQL injection: li todo SQL em src/infrastructure/persistence; 100% parametrizado com ? exceto duas interpolações revisadas (SET LOCAL lock_timeout e WHERE montado por constantes em selectOne); sem achado explorável. Validação: zod strictObject com limites de 128 chars no body de aposta, regex de UUID, Idempotency-Key até 255, limit do ledger com teto 200, cursor validado por re-encode. DoS: body sem limite configurado (padrão do Express, 100 kb), sem rate limit. Headers/CORS: main.ts não configura helmet nem CORS. /metrics, /docs, /docs-json e /health/* públicos. Erros: filtro devolve 500 genérico. Não rodei testes que alterem a infra compartilhada; auditoria só de leitura.

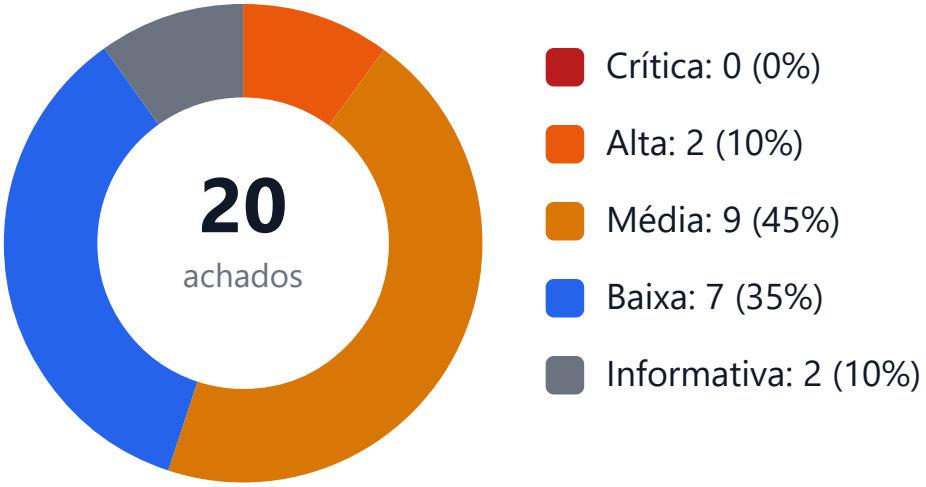
Auditor B: Supply chain: bun audit (v1.4.4) executado, 3 vulnerabilidades (1 alta, 2 moderadas), todas transitivas. Verificado se multer é alcançado: nenhum FileInterceptor/UploadedFile em src. Imagens Docker: todas com tag de versão fixa, nenhuma com latest, nenhuma com digest sha256. Actions do GitHub: ambas fixadas por tag móvel (@v4, @v2), não por SHA; runner ubuntu-latest. Dockerfile sem USER. Portas publicadas no compose. Não rodei testes nem alterei infra compartilhada.

Resumo executivo

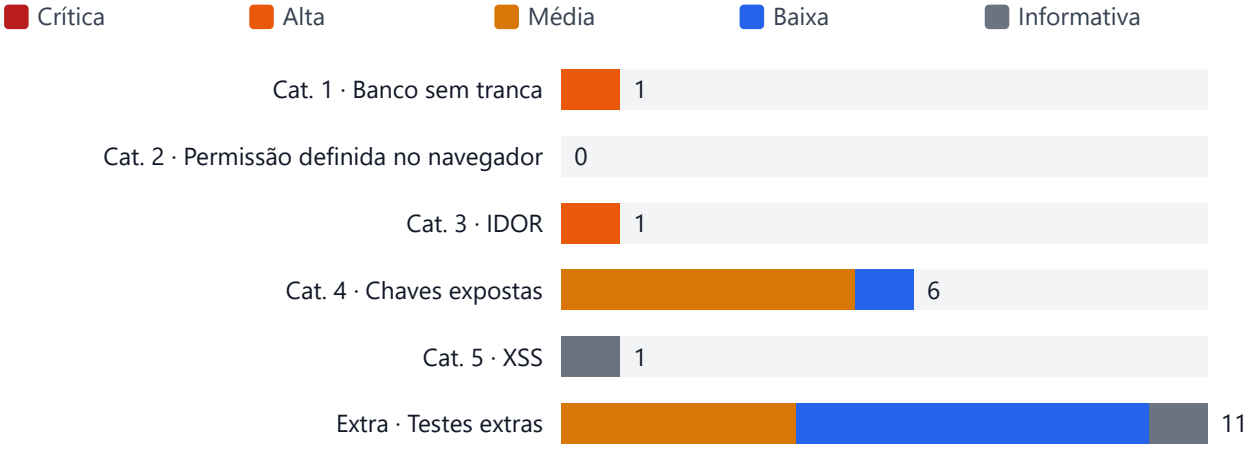
Foram registrados **20** achados (0 crítico(s), 2 alto(s), 9 médio(s), 7 baixo(s), 2 informativo(s)) e **22** pontos fortes. Os **2** achados críticos e altos concentram o risco e devem ser tratados primeiro.

0	2	9	7	2	22
Crítica	Alta	Média	Baixa	Informativa	Pontos fortes

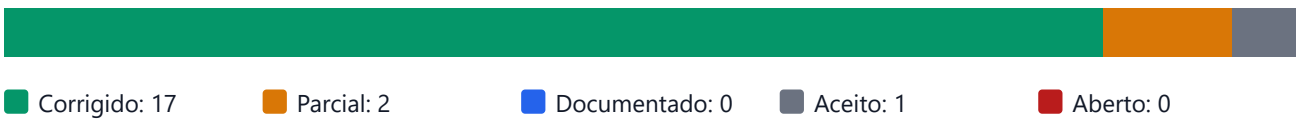
Achados por severidade



Achados por categoria, empilhados por severidade



Achados por status



17 corrigidos, 2 parciais, 0 documentados, 1 aceitos, 0 abertos

Auditor A: 10 achado(s). Auditor B: 10 achado(s).

Pontos fortes

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	Testes extras	Todo SQL de valor é parametrizado com placeholders ?; nenhum dado de requisição é concatenado em SQL. A única interpolação por dado é o SET LOCAL com constante (A-08), e a montagem de WHERE/lock em selectOne usa só literais fixos dos métodos da própria classe.	`src/infrastructure/persistence/repositories/wager-transaction.repository.ts:65-79,119-125; src/infrastructure/persistence/sql.ts:9-22`
Ponto forte	Testes extras	Placeholders também no advisory lock e na query de ledger com cursor; o cursor é validado por regex e por re-encode, impedindo formas alternativas.	`src/infrastructure/persistence/pg-advisory-lock.ts:26,31; src/interfaces/http/cursor.ts:15-22; src/infrastructure/persistence/repositories/ledger.repository.ts:34-38`
Ponto forte	Testes extras	O array de ids em markPublishedMany da outbox é interpolado em literal de array, mas os ids vêm do próprio banco (uuid), não de entrada externa.	`src/infrastructure/persistence/repositories/outbox.repository.ts:57`
Ponto forte	Testes extras	Validação de entrada estrita: zod strictObject (rejeita campos extras), strings de 1-128, UUID por regex, Idempotency-Key de 1-255 sem espaços nas pontas, OPENING bloqueado como kind externo, dinheiro validado por Money.from (sem number).	`src/application/wagering/wager-command.ts:24-47,53-87; src/interfaces/http/wallets.controller.ts:24-28`
Ponto forte	Testes extras	ParseUUIDPipe em todos os IDs de wallet e de transação; limit do ledger validado por regex /^[1-9]\d*$/ e com teto de 200 aplicado duas vezes (controller e query), com leitura de pageSize+1.	`src/interfaces/http/wallets.controller.ts:77,92,115,122-126; src/application/wallet/wallet-queries.ts:23,47`
Ponto forte	Testes extras	Filtro global de exceções devolve 500 genérico e envia stack apenas ao log, nunca o body da request. Não há stack trace nem mensagem de banco na resposta; erros de Postgres são mapeados para códigos fixos.	`src/interfaces/http/http-error.filter.ts:69,76-91; src/infrastructure/persistence/pg-error-mapper.ts:65`
Ponto forte	Banco sem tranca	Isolamento por wallet no ledger e nas consultas de transação por provedor: ledger sempre filtra wallet_id, e lookups por idempotência/id externo usam o par (provider_id, chave), protegido por UNIQUE no banco. Não existem endpoints de listagem global, o que evita vazamento por busca.	`src/infrastructure/persistence/repositories/ledger.repository.ts:34-37; src/infrastructure/persistence/migrations/Migration20261031000001_initial.ts:55-56`

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	Banco sem tranca	O banco impede reatribuir transação a outro provedor: um trigger bloqueia mudança de provider_id (e demais campos imutáveis) em UPDATE.	`src/infrastructure/persistence/migrations/Migration20261031000001_initial.ts:100-104`
Ponto forte	Testes extras	Privilégio mínimo no banco: o runtime usa wagering_app com SELECT/INSERT/UPDATE (sem DELETE, sem DDL); migrations rodam com outro papel.	`infra/postgres/init-roles.sql:1-19; docker-compose.yml:48-50,62`
Ponto forte	Testes extras	Tamanho de payload limitado por padrão: o body-parser do Express aplica 100 kb e todos os campos de texto do POST de aposta têm teto de 128 chars; a HTTP não aceita lote.	`src/application/wagering/wager-command.ts:24-38; src/main.ts:32`
Ponto forte	Testes extras	Interceptor de tracing e logs não incluem body, valores monetários nem headers; spans carregam só identificadores e desfecho.	`src/application/wagering/submit-wager-transaction.ts:43-47; src/interfaces/http/tracing.interceptor.ts:37`
Ponto forte	Permissão definida no navegador	Nenhuma decisão de permissão depende do navegador: não há frontend com papéis, flags ou localStorage. O Swagger UI só descreve a API.	`src/interfaces/http/openapi.ts:12-25; docs/apresentacao/index.html (HTML estático)`
Ponto forte	Chaves expostas	Nenhum segredo real no repositório nem no histórico: sem AKIA, chave privada ou token de API (ghp_, xox, sk-) em 89 commits. O único arquivo .env* versionado é .env.example, que só contém valores de emulador.	`.gitignore:2, git log --all --name-only (apenas .env.example)`
Ponto forte	Chaves expostas	.env está no .gitignore e no .dockerignore, então não vai para o repositório nem para a imagem.	`.gitignore:2; .dockerignore:3`
Ponto forte	Chaves expostas	A aplicação não embute credenciais AWS: o SQSClient usa a cadeia padrão do SDK (IAM role em produção), sem accessKeyId no código de src.	`src/infrastructure/sqs/sqs-queue.ts:33-37`
Ponto forte	Chaves expostas	DATABASE_URL e SQS_ENDPOINT são obrigatórios e validados no startup; não há default com credencial no config.	`src/config/config.ts:31-32`
Ponto forte	Chaves expostas	Separação de papéis no banco (owner roda migrations, app só SELECT/INSERT/UPDATE) reduz o dano de uma credencial de runtime vazada.	`infra/postgres/init-roles.sql:2-3,14-17`
Ponto forte	Chaves expostas	O filtro de erros não vaza mensagens internas nem o body: erros desconhecidos viram 500 'internal error' e o log registra só nome e stack. O logger faz redact de req.body, authorization e valores monetários.	`src/interfaces/http/http-error.filter.ts:69,79-86; src/infrastructure/observability/logger.ts:9-12`

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	XSS	A tooltip do slide 12 usa textContent, não innerHTML, e o único input vindo da URL (location.hash) passa por regex numérica. Não há innerHTML, insertAdjacentHTML, document.write, eval, new Function nem href/src dinâmico na apresentação.	`docs/apresentacao/index.html:1131, 999`
Ponto forte	XSS	O backend só responde JSON ou text/plain; não há template engine nem HTML montado com input do usuário. O Swagger é gerado de DTOs estáticos e os dashboards do Grafana têm só consultas PromQL fixas.	`src/interfaces/http/openapi.ts:12-25; src/interfaces/http/metrics.controller.ts:15; infra/grafana/dashboards/wagering.json`
Ponto forte	XSS	Entrada validada com zod strictObject nos endpoints e no consumer, rejeitando campos extras.	`src/application/wagering/wager-command.ts:27,36; src/interfaces/http/wallets.controller.ts:24-27`
Ponto forte	Testes extras	Dependências com versão exata (exact = true) e instalação com --frozen-lockfile no Docker e no CI; nenhuma imagem usa latest.	`bunfig.toml:2; Dockerfile:6; .github/workflows/ci.yml:17; docker-compose.yml:5,21,31,87,102,110`

Pontos fracos (riscos centrais)

Alta Corrigido

`b427a95` **A-01** · Leitura e reconciliação de wallet, ledger e transação sem autenticação nem checagem de posse (IDOR)

`src/interfaces/http/wallets.controller.ts:71-81, 83-103, 105-119`

GET /wallets/:walletId, GET /wallets/:walletId/ledger, POST /wallets/:walletId/reconciliation e GET /wagering/transactions/:transactionId (wagering.controller.ts:101-111) aceitam qualquer UUID e devolvem saldo, playerId, lançamentos e a transação inteira (idempotencyKey, providerId, externalTransactionId, playerId, roundId, gameId) sem verificar quem pergunta. wallets.controller.ts não tem nem @UseGuards; wagering.controller.ts usa AuthGuard, que retorna true sempre (auth.guard.ts:6-8).

Explorabilidade: Exige conhecer ou adivinhar o UUID. wallet.id vem de IdGenerator (UUID ordenável no tempo, entropia reduzida) e o transactionId/walletId também aparece em respostas, eventos SQS, logs e spans (wager.submit grava wallet.id e provider.id). Explorável por qualquer cliente com acesso de rede à porta 3001-3003 (bind 0.0.0.0, docker-compose.yml:70-71). No cenário do desafio (sem exposição pública) o risco é baixo; exposto em rede compartilhada ou internet, qualquer par com um id lê dados financeiros de outro.

Alta Corrigido

b427a95 **A-02** · providerId é autodeclarado e a wallet não pertence a nenhum provedor: qualquer chamador movimenta qualquer wallet

`src/application/wagering/submit-wager-transaction.ts:93-94`

O comando traz providerId, walletId e playerId do body (wager-command.ts:26-38). A única checagem entre wallet e dono é tx.playerId === wallet.playerId (decide-wager.ts:43); não há vínculo wallet-provedor (wallets sem provider_id) e NoopProviderIdentity.assertCanActAs (auth.guard.ts:20-22) nunca é invocado. O mesmo vale nas consultas por provedor: GET /providers/:providerId/wagering/transactions/:externalTransactionId (wagering.controller.ts:113-126) filtra por provider_id (wager-transaction.repository.ts:77-79), mas o valor é escolhido pelo próprio chamador.

Explorabilidade: Quem conhece walletId + playerId (ambos UUID, devolvidos por GET /wallets/:id e ledger sem auth, ver A-01) envia BET/WIN/REFUND com qualquer providerId e altera saldo da wallet. Condição: serviço alcançável pela rede; não precisa de credencial. Também permite ocupar chaves (provider_id, idempotency_key/external_transaction_id) de outro provedor e causar IDEMPOTENCY_CONFLICT ou EXTERNAL_ID_CONFLICT para o dono real.

Achados por categoria

Categoria 1 — Banco sem tranca

Severidade	Status	Arquivo:linha	Descrição
Alta	Corrigido b427a95	`src/application/wagering/submit-wager-transaction.ts:93-94`	**A-02 — providerId é autodeclarado e a wallet não pertence a nenhum provedor: qualquer chamador movimenta qualquer wallet** O comando traz providerId, walletId e playerId do body (wager-command.ts:26-38). A única checagem entre wallet e dono é tx.playerId === wallet.playerId (decide-wager.ts:43); não há vínculo wallet-provedor (wallets sem provider_id) e NoopProviderIdentity.assertCanActAs (auth.guard.ts:20-22) nunca é invocado. O mesmo vale nas consultas por provedor: GET /providers/:providerId/wagering/transactions/:externalTransactionId (wagering.controller.ts:113-126) filtra por provider_id (wager-transaction.repository.ts:77-79), mas o valor é escolhido pelo próprio chamador.

Categoria 2 — Permissão definida no navegador

Nenhum achado nesta categoria.

Categoria 3 — IDOR

Severidade	Status	Arquivo:linha	Descrição
Alta	Corrigido b427a95	`src/interfaces/http/wallets.controller.ts:71-81, 83-103, 105-119`	**A-01 — Leitura e reconciliação de wallet, ledger e transação sem autenticação nem checagem de posse (IDOR)** GET /wallets/:walletId, GET /wallets/:walletId/ledger, POST /wallets/:walletId/reconciliation e GET /wagering/transactions/:transactionId (wagering.controller.ts:101-111) aceitam qualquer UUID e devolvem saldo, playerId, lançamentos e a transação inteira (idempotencyKey, providerId, externalTransactionId, playerId, roundId, gameId) sem verificar quem pergunta. wallets.controller.ts não tem nem @UseGuards; wagering.controller.ts usa AuthGuard, que retorna true sempre (auth.guard.ts:6-8).

Categoria 4 — Chaves expostas

Severidade	Status	Arquivo:linha	Descrição
Média	Corrigido 71bff07	`docker-compose.yml:5-11`	**B-01 — Superusuário postgres/postgres com porta publicada em todas as interfaces** O superusuário do cluster usa senha trivial e a porta 5432 é publicada sem prefixo 127.0.0.1, então escuta em 0.0.0.0 do host. O mesmo par aparece em docker-compose.test.yml:8 (porta 5433), test/helpers/env.ts:17 e test/integration/db-outage.test.ts:20,53.
Média	Corrigido 71bff07	`infra/postgres/init-roles.sql:5-6`	**B-02 — Papéis wagering_owner/owner e wagering_app/app com senha trivial, versionados em vários arquivos** As senhas dos dois papéis são literais no SQL de init e repetidas em docker-compose.yml:50 (MIGRATION_DATABASE_URL ...wagering_owner:owner@...), docker-compose.yml:62 (DATABASE_URL ...wagering_app:app@...), .env.example:9, mikro-orm.config.ts:12 (LOCAL_OWNER_URL), test/helpers/env.ts:14-15 e testes.
Média	Corrigido b427a95	`src/config/config.ts:29-33`	**B-03 — Nenhuma validação no startup rejeita credenciais padrão ou configuração insegura em produção** DATABASE_URL só exige string não vazia e SQS_ENDPOINT só exige URL válida. Não existe NODE_ENV nem checagem que recuse postgres://wagering_app:app@..., usuário postgres, endpoint SQS sem TLS ou chaves AWS 'test'. As chaves AWS nem são lidas pelo config: o SDK as resolve sozinho (sqs-queue.ts:33-37), então a aplicação sobe com qualquer valor. mikro-orm.config.ts:40 ainda cai para LOCAL_OWNER_URL (credencial owner) se MIGRATION_DATABASE_URL e DATABASE_URL faltarem.
Média	Corrigido 71bff07	`docker-compose.yml:109-118`	**B-04 — Grafana com admin/admin e porta publicada (perfil observability)** Admin padrão admin/admin, documentado no README.md:60, e acesso anônimo como Viewer, com a porta publicada em 0.0.0.0.
Média	Corrigido 71bff07	`docker-compose.yml:20-23, 63-66`	**B-05 — Chaves AWS 'test/test' e MiniStack sem autenticação com porta publicada** O emulador SQS aceita qualquer credencial, e as chaves test/test ficam em docker-compose.yml:35-36 e 65-66, .env.example:12-13, scripts/load-test.ts:291,725, test/helpers/sqs.ts:20 e README.md:280. A porta 4566 é publicada em 0.0.0.0.

Severidade	Status	Arquivo:linha	Descrição
Baixa	Corrigido 71bff07	`docker-compose.yml:86-107`	**B-09 — Prometheus, Jaeger e /metrics da API sem autenticação** As UIs do Prometheus (9090) e do Jaeger (16686) ficam em 0.0.0.0 sem auth, e GET /metrics (src/interfaces/http/metrics.controller.ts:6-17) é público na porta da API. Jaeger guarda traces com IDs de transação e carteira.

Categoria 5 — XSS

Severidade	Status	Arquivo:linha	Descrição
Informativa	Parcial e3f86d9	`src/application/wagering/wager-command.ts:27, 68-73`	**B-10 — Erros de validação ecoam nomes de chaves enviadas pelo cliente, sem nosniff** strictObject faz o zod responder com 'Unrecognized key' mais a chave enviada, e http-error.filter.ts:90 devolve isso em error.message. O mesmo padrão existe em wallets.controller.ts:57-59 e wager-consumer.ts:63-65. A resposta é application/json (res.json) e não há middleware helmet, então X-Content-Type-Options: nosniff não é enviado. **Situação:** X-Content-Type-Options: nosniff corrigido via helmet. Limite de tamanho das chaves ecoadas em wager-command.ts não implementado.

Categoria extra — Testes extras

Severidade	Status	Arquivo:linha	Descrição
Média	Corrigido b427a95	`src/interfaces/http/auth.guard.ts:3-9`	**A-03 — Ausência total de autenticação (AuthGuard no-op) em todas as rotas, incluindo escrita** Nenhuma rota exige identidade. POST /wallets (wallets.controller.ts:39-69) cria wallets com saldo inicial arbitrário e não tem sequer o guard aplicado; POST /wagering/transactions altera saldo. A decisão é documentada (ARCHITECTURE §14), logo é achado de contexto: a raiz de A-01 e A-02.
Média	Corrigido b427a95	`src/application/wallet/reconcile-wallet.ts:26-33`	**A-04 — Endpoint de reconciliação público executa agregação sobre todo o ledger com FOR SHARE (amplificação de DoS)** POST /wallets/:walletId/reconciliation (wallets.controller.ts:105-119) é público, sem rate limit, e a cada chamada faz SUM/COUNT sobre todas as linhas de wallet_ledger_entries da wallet (ledger.repository.ts:54-63) dentro de uma transação que mantém FOR SHARE na wallet. O FOR SHARE conflita com o FOR UPDATE de apostas (submit-wager-transaction.ts:94).

Severidade	Status	Arquivo:linha	Descrição
Média	Corrigido e3f86d9	`src/main.ts:32-38`	**A-05 — Sem rate limiting; limite de body e de lote implícitos (padrão do framework)** O bootstrap não registra throttler, não define limite de body (vale o padrão de 100 kb do body-parser do Express, não declarado) nem timeouts do servidor HTTP (headersTimeout/requestTimeout padrão do Node). package.json não contém @nestjs/throttler. Cada POST /wagering/transactions abre uma transação com lock de wallet; Idempotency-Key e X-Correlation-Id só são limitados pelo teto de header do Node. **Situação:** Limite de body de 64 kb corrigido (413 PAYLOAD_TOO_LARGE). Rate limit não implementado: documentado (ARCHITECTURE §18, README) como responsabilidade da borda. Timeouts do servidor HTTP não alterados.
Média	Corrigido cff69c3	`bun.lock:n/a (dependência transitiva de @nestjs/platform-express)`	**B-06 — bun audit: multer 2.4.0 com DoS de severidade alta (transitivo, não alcançado)** bun audit reporta 3 vulnerabilidades: multer 2.4.0 (alta, GHSA-q7qp-rjg7-q26m), ajv 8.13.0 (moderada, ReDoS com $data, GHSA-2g4f-4pwh-qvx6, via @mikro-orm/migrations > umzug, só no CLI de migration) e js-yaml 5.3.0 (moderada, GHSA-r3ph-w7gj-g6xm, via @nestjs/swagger).
Baixa	Corrigido e3f86d9	`src/main.ts:30-38`	**A-06 — Sem helmet: faltam headers de segurança e X-Powered-By fica exposto; CORS não configurado** main.ts não usa helmet nem app.enableCors; o Express envia X-Powered-By: Express e nenhum X-Content-Type-Options, Strict-Transport-Security ou CSP. O CORS não habilitado é seguro por padrão (sem Access-Control-Allow-Origin, o navegador bloqueia origens cruzadas), então não é achado por si só. O Swagger UI em /docs é servido sob essa mesma ausência de CSP.

Severidade	Status	Arquivo:linha	Descrição
Baixa	Parcial e3f86d9	`src/interfaces/http/metrics.controller.ts:5-17`	**A-07 — /metrics, /docs, /docs-json e /health/ready públicos, sem autenticação, nas mesmas portas da API** Métricas Prometheus, o documento OpenAPI (openapi.ts:24, /docs e /docs-json) e a readiness são servidos na porta pública da API (docker-compose.yml:70-71), sem guard. As métricas só usam labels de baixa cardinalidade (status, kind, source, result, component; prom-metrics.ts:13,20,27,65), sem identificadores de jogador ou provedor. O ready devolve o estado de dependências (health.controller.ts:63-65, mensagem 'not ready: db=... sqs=...'), pelo filtro de erro (http-error.filter.ts:56-62). **Situação:** DOCS_ENABLED implementado e documentado no README. /metrics e /health/ready continuam na porta da API, e o corpo do ready ainda lista db/sqs.
Baixa	Corrigido e3f86d9	`src/infrastructure/persistence/mikro-unit-of-work.ts:30`	**A-08 — SET LOCAL lock_timeout monta SQL por interpolação de string** SET não aceita parâmetro ligado, então o valor é interpolado. Hoje não é explorável: lockTimeout vem do default 'const DEFAULT_LOCK_TIMEOUT = "5s"' (linha 14) porque persistence.module.ts:44 constrói `new MikroUnitOfWork(orm.em, undefined, metrics)`, e o construtor é público com parâmetro string livre. Se alguém ligar o valor a env ou a entrada externa, vira injeção SQL (aspas simples) executada com o papel wagering_app.
Baixa	Corrigido b427a95	`src/interfaces/http/wagering.controller.ts:88-93`	**A-09 — X-Correlation-Id e providerId/externalTransactionId sem formato restrito; correlationId sem limite próprio** O header X-Correlation-Id é repassado sem validação de formato ou tamanho para o payload dos eventos de outbox (submit-wager-transaction.ts:188; record-permanent-failure.ts:104) e, portanto, ao SQS. O mesmo vale para o do POST /wallets (wallets.controller.ts:53,67). providerId e externalTransactionId aceitam qualquer string de 1-128 chars (wager-command.ts:24,28-29), inclusive controle e Unicode, e os params do GET por externalId nem têm limite (wagering.controller.ts:120-121). Logs usam pino (JSON), então não há injeção de linha de log; o risco é dado sujo em eventos consumidos por terceiros.

Severidade	Status	Arquivo:linha	Descrição
Baixa	Corrigido 71bff07	`.github/workflows/ci.yml:7, 10-12`	**B-07 — Actions do GitHub fixadas por tag móvel e runner ubuntu-latest** As duas actions usam tags (@v4, @v2) que o mantenedor pode mover. O workflow não declara permissions, então o GITHUB_TOKEN herda o padrão do repositório.
Baixa	Corrigido 71bff07	`Dockerfile:1, 3-12`	**B-08 — Imagens Docker com tag de versão mas sem digest; contêiner da aplicação roda como root** Nenhuma imagem usa latest (oven/bun:1.4.4-alpine, postgres:17.6-alpine, ministackorg/ministack:1.5.35, amazon/aws-cli:2.17.0, prom/prometheus:v3.5.0, jaegertracing/all-in-one:1.62.0, grafana/grafana:12.1.0), mas nenhuma tem digest sha256. O Dockerfile não define USER, então a app roda como root, e COPY . . leva test/, docs/ e scripts/ para a imagem. A imagem instala devDependencies (bun install sem --production).
Informativa	Aceito	`src/interfaces/http/http-error.filter.ts:64-67`	**A-10 — Mensagens de erro de HttpException do Nest e de zod são devolvidas ao cliente** Mensagens de validação (ex.: 'Validation failed (uuid is expected)', erros zod com caminho do campo) e de rota inexistente ('Cannot GET /x') chegam ao cliente. Isso é esperado e não contém stack, SQL nem caminho de arquivo. Todo erro não mapeado vira 500 com 'internal error' (linha 69) e o stack só vai para o log (linhas 79-86). Exceção: NotFoundError em ledger.repository.ts:66 inclui o walletId, que é entrada do próprio chamador.

Recomendações priorizadas

P1 — crítica e alta: corrigir antes de qualquer exposição

1. Alta Corrigido
   `b427a95` **A-01** Leitura e reconciliação de wallet, ledger e transação sem autenticação nem checagem de posse (IDOR)
   Introduzir autenticação (API key ou JWT por provedor) e verificar que a wallet/transação pertence ao provedor autenticado, via coluna provider_id em wallets ou join com wager_transactions. Retornar 404 (não 403) quando o recurso é de outro provedor. Se a decisão do escopo for manter sem auth, documentar o serviço como interno e restringir a rede (bind em loopback ou rede interna do compose).
2. Alta Corrigido
   `b427a95` **A-02** providerId é autodeclarado e a wallet não pertence a nenhum provedor: qualquer chamador movimenta qualquer wallet
   Autenticar o provedor e derivar providerId da identidade (ignorar ou comparar o do body), chamando ProviderIdentityPort.assertCanActAs. Registrar provider_id em wallets na criação e rejeitar com WALLET_NOT_FOUND quando a wallet for de outro provedor. Aplicar a mesma regra no consumer SQS (identidade por fila ou atributo assinado).

P2 — média: corrigir no próximo ciclo

1. Média Corrigido
   `b427a95` **A-03** Ausência total de autenticação (AuthGuard no-op) em todas as rotas, incluindo escrita
   Aplicar um guard real (API key por provedor com comparação em tempo constante, ou JWT) globalmente via APP_GUARD, com lista explícita de rotas públicas (health). Se mantiver no-op por escopo, remover a falsa sensação de proteção (o decorator existe só em wagering.controller) e documentar no README que a rede é a fronteira de confiança.
2. Média Corrigido
   `b427a95` **A-04** Endpoint de reconciliação público executa agregação sobre todo o ledger com FOR SHARE (amplificação de DoS)
   Proteger o endpoint por autenticação administrativa e aplicar rate limit; avaliar reconciliar por snapshot (REPEATABLE READ) em vez de FOR SHARE para não bloquear escrita.
3. Média Corrigido
   `e3f86d9` **A-05** Sem rate limiting; limite de body e de lote implícitos (padrão do framework)
   Adicionar @nestjs/throttler (por IP e, depois, por provedor), declarar explicitamente o limite de body (NestFactory.create com bodyParser opções ou app.useBodyParser('json', { limit: '16kb' })) e configurar server.requestTimeout/headersTimeout.
4. Média Corrigido
   `71bff07` **B-01** Superusuário postgres/postgres com porta publicada em todas as interfaces
   Publicar com 127.0.0.1:${POSTGRES_PORT:-5432}:5432, ler a senha de uma variável obrigatória sem default (${POSTGRES_PASSWORD:?defina}) e documentar a geração no README.

5. Média Corrigido
   `71bff07` **B-02** Papéis wagering_owner/owner e wagering_app/app com senha trivial, versionados em vários arquivos
   Gerar as senhas por variável de ambiente no init (script .sh com envsubst ou psql -v) e propagar via compose com ${VAR:?}. Manter os valores fracos só nos arquivos de teste (docker-compose.test.yml).
6. Média Corrigido
   `b427a95` **B-03** Nenhuma validação no startup rejeita credenciais padrão ou configuração insegura em produção
   Adicionar NODE_ENV (ou APP_ENV) ao schema; em production, rejeitar via superRefine senhas conhecidas (app, owner, postgres, test), usuário postgres e SQS_ENDPOINT sem https. Remover o fallback LOCAL_OWNER_URL quando NODE_ENV=production.
7. Média Corrigido
   `71bff07` **B-04** Grafana com admin/admin e porta publicada (perfil observability)
   Usar GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD:?}, publicar em 127.0.0.1 e manter o anônimo desligado fora de demo.
8. Média Corrigido
   `71bff07` **B-05** Chaves AWS 'test/test' e MiniStack sem autenticação com porta publicada
   Publicar 4566 em 127.0.0.1. Documentar que em AWS real o consumer usa IAM role e que as chaves test só valem no emulador.
9. Média Corrigido
   `cff69c3` **B-06** bun audit: multer 2.4.0 com DoS de severidade alta (transitivo, não alcançado)
   Rodar bun audit fix e, se necessário, usar overrides para multer >=2.5.0, ajv >=8.18.0 e js-yaml acima de 5.4.0. Incluir bun audit no CI.

P3 — baixa e informativa: melhoria e higiene

1. Baixa Corrigido
   `e3f86d9` **A-06** Sem helmet: faltam headers de segurança e X-Powered-By fica exposto; CORS não configurado
   Adicionar helmet (com CSP compatível com o Swagger UI) e desabilitar x-powered-by (app.getHttpAdapter().getInstance().disable('x-powered-by')). Manter CORS desligado; se for preciso, usar lista explícita de origens.
2. Baixa Parcial
   `e3f86d9` **A-07** /metrics, /docs, /docs-json e /health/ready públicos, sem autenticação, nas mesmas portas da API
   Servir /metrics em porta/host interno separado (ou exigir token de scraping) e desabilitar /docs fora de ambiente de desenvolvimento via flag de config. Reduzir a mensagem do ready para 'not ready' e manter o detalhe só em log.
3. Baixa Corrigido
   `e3f86d9` **A-08** SET LOCAL lock_timeout monta SQL por interpolação de string
   Validar o parâmetro no construtor (regex /^\d+(ms|s|min)$/) ou receber número de milissegundos e interpolar Number.isInteger(value) validado; alternativa: `SELECT set_config('lock_timeout', ?, true)` com parâmetro ligado.

4. Baixa Corrigido
   `b427a95` **A-09** X-Correlation-Id e providerId/externalTransactionId sem formato restrito; correlationId sem limite próprio
   Validar X-Correlation-Id com regex /^[A-Za-z0-9._:-]{1,128}$/ (ou descartar e gerar UUID); aplicar o mesmo conjunto de caracteres em providerId e externalTransactionId e limitar os params do GET a 128 chars.
5. Baixa Corrigido
   `71bff07` **B-07** Actions do GitHub fixadas por tag móvel e runner ubuntu-latest
   Fixar por SHA de commit com comentário da versão, adicionar permissions: contents: read no topo e fixar o runner (ubuntu-24.04).
6. Baixa Corrigido
   `71bff07` **B-08** Imagens Docker com tag de versão mas sem digest; contêiner da aplicação roda como root
   Adicionar USER bun (a imagem oficial já tem), bun install --production --frozen-lockfile em estágio de runtime, e fixar digests dos serviços de produção.
7. Baixa Corrigido
   `71bff07` **B-09** Prometheus, Jaeger e /metrics da API sem autenticação
   Publicar em 127.0.0.1 e restringir /metrics a rede interna (porta separada ou proxy).
8. Informativa Aceito **A-10** Mensagens de erro de HttpException do Nest e de zod são devolvidas ao cliente
   Nenhuma obrigatória. Opcional: padronizar a mensagem de HTTP_ERROR para texto fixo.
9. Informativa Parcial
   `e3f86d9` **B-10** Erros de validação ecoam nomes de chaves enviadas pelo cliente, sem nosniff
   Opcional: limitar o tamanho das chaves ecoadas e adicionar helmet (nosniff) em main.ts. Qualquer consumidor deve usar textContent.

ISSUES PARA O GITHUB

Cada bloco abaixo é uma issue completa em Markdown, somente para achados **aberto** ou **parcial**. Achados informativos sem ação não viram issue. Achados com o mesmo `issueGroup` ficam numa única issue; membros já corrigidos ficam fora dela.

````
--- ISSUE 1 ---
# [Segurança] http-hardening

**Labels:** `security`, `severidade:baixa`

### A-07 — /metrics, /docs, /docs-json e /health/ready públicos, sem autenticação, nas mesmas portas da API

**Categoria:** extra — Testes extras
**Severidade:** Baixa
**Status:** Parcial — DOCS_ENABLED implementado e documentado no README. /metrics e /health/ready continuam na porta da API, e o corpo do ready ainda lista db/sqs.

**Descrição:** Métricas Prometheus, o documento OpenAPI (openapi.ts:24, /docs e /docs-json) e a readiness são servidos na porta pública da API (docker-compose.yml:70-71), sem guard. As métricas só usam labels de baixa cardinalidade (status, kind, source, result, component; prom-metrics.ts:13,20,27,65), sem identificadores de jogador ou provedor. O ready devolve o estado de dependências (health.controller.ts:63-65, mensagem 'not ready: db=... sqs=...'), pelo filtro de erro (http-error.filter.ts:56-62).

**Por que é explorável:** Qualquer cliente de rede lê volumes e taxas de erro (inteligência operacional, contadores de reconciliationMismatch), o contrato completo da API e o estado de Postgres/SQS. Sem dado pessoal ou segredo, o impacto é de reconhecimento. Condição: porta publicada.

**Evidência:** `src/interfaces/http/metrics.controller.ts:5-17`

```
@ApiTags("ops")
@Controller("metrics")
export class MetricsController {
  @Get()
  async get(): Promise<string> {
    return this.metrics.registry.metrics();
  }
}
```

**Impacto:** Reconhecimento para ataques; divulgação de saúde de dependências.

**Sugestão de correção:** Servir /metrics em porta/host interno separado (ou exigir token de scraping) e desabilitar /docs fora de ambiente de desenvolvimento via flag de config. Reduzir a mensagem do ready para 'not ready' e manter o detalhe só em log.

## Critérios de aceite

- [ ] /metrics não responde na porta pública da API
- [ ] /docs depende de uma flag (ex.: ENABLE_DOCS) desligada por padrão em produção
- [ ] Corpo de /health/ready em erro não lista db/sqs
--- FIM ISSUE 1 ---
````

```
--- ISSUE 2 ---
# [Segurança] headers-http

**Labels:** `security`, `severidade:informativa`

### B-10 — Erros de validação ecoam nomes de chaves enviadas pelo cliente, sem nosniff

**Categoria:** 5 — XSS
**Severidade:** Informativa
**Status:** Parcial — X-Content-Type-Options: nosniff corrigido via helmet. Limite de tamanho das chaves ecoadas em wager-command.ts não implementado.

**Descrição:** strictObject faz o zod responder com 'Unrecognized key' mais a chave enviada, e http-error.filter.ts:90 devolve isso em error.message. O mesmo padrão existe em wallets.controller.ts:57-59 e wager-consumer.ts:63-65. A resposta é application/json (res.json) e não há middleware helmet, então X-Content-Type-Options: nosniff não é enviado.

**Por que é explorável:** Para XSS seria preciso um cliente que injete o campo message em HTML sem escapar, o que não existe neste repositório. Navegadores não renderizam application/json como HTML, e um POST JSON cross-origin não é forjável por formulário. Não explorável hoje; é apenas reflexão de input.

**Evidência:** `src/application/wagering/wager-command.ts:27, 68-73`

```
.strictObject({ ... })
...
const details = parsed.error.issues
      .map((i) => `${i.path.join(".") || "body"}: ${i.message}`)
      .join("; ");
throw new InvalidPayloadError(`invalid payload: ${details}`);
```

Impacto: Nenhum hoje. Se algum painel futuro renderizar error.message com innerHTML, vira XSS refletido.

Sugestão de correção: Opcional: limitar o tamanho das chaves ecoadas e adicionar helmet (nosniff) em main.ts. Qualquer consumidor deve usar textContent.

Critérios de aceite

- [ ] resposta 400 com chave inesperada não excede um limite de tamanho
- [ ] cabeçalho X-Content-Type-Options: nosniff presente
--- FIM ISSUE 2 ---
```

Já corrigidos

- b427a95 **A-01** Leitura e reconciliação de wallet, ledger e transação sem autenticação nem checagem de posse (IDOR)
- b427a95 **A-02** providerId é autodeclarado e a wallet não pertence a nenhum provedor: qualquer chamador movimenta qualquer wallet
- b427a95 **A-03** Ausência total de autenticação (AuthGuard no-op) em todas as rotas, incluindo escrita
- b427a95 **A-04** Endpoint de reconciliação público executa agregação sobre todo o ledger com FOR SHARE (amplificação de DoS)

- `e3f86d9` **A-05** Sem rate limiting; limite de body e de lote implícitos (padrão do framework)
- `71bff07` **B-01** Superusuário postgres/postgres com porta publicada em todas as interfaces
- `71bff07` **B-02** Papéis wagering_owner/owner e wagering_app/app com senha trivial, versionados em vários arquivos
- `b427a95` **B-03** Nenhuma validação no startup rejeita credenciais padrão ou configuração insegura em produção
- `71bff07` **B-04** Grafana com admin/admin e porta publicada (perfil observability)
- `71bff07` **B-05** Chaves AWS 'test/test' e MiniStack sem autenticação com porta publicada
- `cff69c3` **B-06** bun audit: multer 2.4.0 com DoS de severidade alta (transitivo, não alcançado)
- `e3f86d9` **A-06** Sem helmet: faltam headers de segurança e X-Powered-By fica exposto; CORS não configurado
- `e3f86d9` **A-08** SET LOCAL lock_timeout monta SQL por interpolação de string
- `b427a95` **A-09** X-Correlation-Id e providerId/externalTransactionId sem formato restrito; correlationId sem limite próprio
- `71bff07` **B-07** Actions do GitHub fixadas por tag móvel e runner ubuntu-latest
- `71bff07` **B-08** Imagens Docker com tag de versão mas sem digest; contêiner da aplicação roda como root
- `71bff07` **B-09** Prometheus, Jaeger e /metrics da API sem autenticação